

1. Balanced Expression with Custom Rules

Write a C program to check whether an expression is balanced.

Extend the logic to:

- Support {}, [], ()
- Ignore characters inside quotes (" " or ' ')

2. Stack-Based Undo/Redo System

Implement a text editor simulation using stacks:

- UNDO reverses last operation
- REDO reapplies undone operation

Use two stacks to manage operations.

3. Next Greater Element

Given an array, find the **next greater element** for each element using stack.

Example:

[4, 5, 2, 10] → [5, 10, 10, -1]

4. Infix to Postfix with Evaluation

Convert an infix expression to postfix and then evaluate it using stack.

Example:

$(3+5)*2 \rightarrow 3\ 5\ +\ 2\ *\rightarrow 16$

5. Stock Span Problem

For a given array of stock prices, calculate span of each day using stack.

6. Circular Queue Implementation

Implement a circular queue using array:

- Detect overflow & underflow
- Support wrap-around

7. Priority Queue using Linked List

Implement a priority queue where:

- Higher priority elements are dequeued first
- Maintain sorted insertion

8. Print Binary Numbers from 1 to N

Use queue to generate binary numbers from 1 to N .

Example:

$N=5 \rightarrow 1, 10, 11, 100, 101$

9. Queue using Two Stacks

Implement queue operations (enqueue , dequeue) using two stacks.

10. CPU Scheduling Simulation (FCFS)

Simulate First Come First Serve scheduling:

- Input: process ID, burst time
- Output: waiting time, turnaround time

11. Reverse Linked List in Groups of K

Reverse nodes in groups of size k .

Example:

1→2→3→4→5 , k=2 → 2→1→4→3→5

12. Detect and Remove Loop

Detect a loop using Floyd's Cycle Detection and remove it.

13. Merge Two Sorted Linked Lists

Merge two sorted linked lists into one sorted list.

14. Find Middle Without Length

Find the middle node using slow and fast pointers.

15. Clone Linked List with Random Pointer

Each node has:

- next
- random pointer

Clone the entire list.

16. Reverse Doubly Linked List

Reverse a doubly linked list by modifying links.

17. LRU Cache Implementation

Use doubly linked list + hash map to implement LRU Cache:

- Capacity-based eviction
- $O(1)$ operations

18. Insert in Sorted Doubly Linked List

Insert a node while maintaining sorted order.

19. Delete All Occurrences of Key

Delete all nodes with a given value from a doubly linked list.

20. Convert DLL to Binary Tree (Level Order)

Convert a doubly linked list into a binary tree in level order.